

Apuntes_NumPy_2_Arrays_Operaciones

May 11, 2026

1 Apuntes NumPy 2: Como podemos Operar con los Arrays

Una vez visto que es Numpy y sus objetos principales, los arrays, que características de comportamiento tienen los array (*indexar, iterar y modificar*), y que metodos se pueden utilizar con ellos, ahora toca conocer los diferentes calculos que permite la libreria para sacarle partido **a la gestion de datos y el calculo computarizado**

Veremos:

- **Asignadores**
- **Operaciones Matematicas**
- **Operaciones de agregacion y estadisticas**
- **Algebra de vectores y matrices**

1.1 Operaciones basicas: Asignaciones

El primero que veremos es *el asignador habitual*, permite estos cambios:

- **Modificar un elemento concreto** de un array

“nombre array”[“indice fila”, “indice columna”] = “valor nuevo”

```
[34]: import numpy as np

my_array = np.arange( 0,6).reshape (2,3)
my_array[1,1] = 50
print(my_array)
```

```
[[ 0  1  2]
 [ 3 50  5]]
```

- **Modificar una rebanada con un conjunto de valores** de un array

“nombre array”[“indice fila”, “indice columna” :] = “[valores nuevos]”

```
[24]: import numpy as np

my_array = np.arange( 0,15).reshape (3,5)
my_array[0,2:] = [20,450,43]
print(my_array)
```

```
[[ 0  1 20 450 43]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

TIP: El n° de elementos que incluyes tiene que ser igual al **numero de elementos** que completan el array desde el indice que le indicas sino dara error

```
[25]: import numpy as np

my_array = np.arange( 0,15).reshape (3,5)
my_array[0,1:] = [20,450,43]
print(my_array)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[25], line 4
      1 import numpy as np
      3 my_array = np.arange( 0,15).reshape (3,5)
----> 4 my_array[0,1:] = [20,450,43]
      5 print(my_array)

ValueError: could not broadcast input array from shape (3,) into shape (4,)
```

- Modificar todos los valores de las filas y/o las columnas por un mismo valor de un array

“nombre array”[“indice fila” : , “indice columna” :] = “valor nuevo”

```
[29]: import numpy as np

my_array = np.arange( 0,15).reshape (3,5)
my_array[1: ,3:] = 99
print(my_array)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7 99 99]
 [10 11 12 99 99]]
```

1.2 Operaciones Matematicas

Como ya hemos comentado anteriormente los Array permiten entre otras cosas realizar operaciones sencillas matematicas sobre la misma estructura lo que posibilita un amplio uso en Python.

Ahora iremos viendo posibilidades:

1.2.1 Operaciones con Arrays de misma forma

La forma mas sencilla de operar con arrays es que dispongan de una misma estructura matricial, ya sean de 1 dimensiones o de mas.

Si disponen la misma forma existen diferentes operaciones ue se pueden realizar:

- Sumar un numero natural a todos los elementos del array
- Multiplicar un numero natural a todos los elementos del array
- Sumar 2 arrays elemento a elemento
- Restar 2 arrays elemento a elemento
- Dividir 2 arrays elemento a elemento
- Multiplicar 2 arrays elemento a elemento.
- Podemos realizar **sumar un numero directamente a todos los elementos de un array**

```
[39]: import numpy as np

my_array = np.arange( 4,12).reshape (4,2)
my_array_suma= my_array +1
print(my_array)
print(my_array_suma)
```

```
[[ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
[[ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
```

- Podemos realizar **multiplicar por un scalar un array de 1 o varias dimensiones**

No confundir con **poner en una potencia**

```
[43]: import numpy as np

my_array = np.arange( 4,12).reshape (4,2)
my_array_multi = my_array * 50
print(my_array_multi)
```

```
[[200 250]
 [300 350]
 [400 450]
 [500 550]]
```

- Podemos realizar **sumas de 2 arrays elemento a elemento**

```
[40]: import numpy as np

my_array = np.arange( 4,12).reshape (4,2)
my_array_suma = my_array +1
my_array_sumatorio = my_array + my_array_suma
print(my_array_sumatorio)
```

```
[[ 9 11]
 [13 15]
 [17 19]
 [21 23]]
```

- Podemos realizar **restas de 2 arrays elemento a elemento**

```
[42]: import numpy as np

my_array = np.arange( 4,12).reshape (4,2)
my_array_suma = my_array +1
my_array_restado = my_array - my_array_suma
print(my_array_restado)
```

```
[[ -1 -1]
 [ -1 -1]
 [ -1 -1]
 [ -1 -1]]
```

- Podemos realizar **divisiones de 2 arrays elemento a elemento**

```
[13]: import numpy as np

my_array1 = np.arange( 10,20).reshape (5,2)
my_array2 = np.array ([4, 5])
my_array_div = my_array1 / my_array2
print(my_array_div)
```

```
[[2.5 2.2]
 [3.  2.6]
 [3.5 3.  ]
 [4.  3.4]
 [4.5 3.8]]
```

- Podemos realizar **multiplicaciones de 2 arrays elemento a elemento**

```
[14]: import numpy as np

my_array1 = np.arange( 10,20).reshape (5,2)
my_array2 = np.array ([4, 5])
my_array_div = my_array1 * my_array2
print(my_array_div)
```

```
[[40 55]
 [48 65]
 [56 75]
 [64 85]
 [72 95]]
```

1.2.2 Operaciones con Arrays de distinto tamaño

Es posible operar con arrays de distintos tamaños, **cumpliendo que la dimensión del menor array tenga un tamaño compatible con el array de más dimensiones**

Cuando mezclamos dos arrays con diferente número de dimensiones en operaciones elemento a elemento, NumPy automáticamente toma el array de dimensión menor y replica la operación con cada dimensión del array mayor, como si fuera un bucle.

Se pueden realizar las mismas operaciones con 2 arrays del mismo tamaño, pero con la diferencia que **los elementos operan con los de su índice correspondiente (misma fila)**

En definitiva es como si pusieramos un *.vstack* en el array de menor dimensión, tantas veces como dimensiones tenga el mayor y realizáramos la operación por cada iteración.

Este mecanismo es conocido como **propagación o broadcasting**.

```
[12]: import numpy as np

my_matriz_2d = np.arange( 2,6).reshape (2,2)
my_vector_1d = np.array ([4, 5])
my_suma_doble_array = my_matriz_2d + my_vector_1d
print(my_matriz_2d)
print ( 40*"-")
print(my_suma_doble_array)
```

```
[[2 3]
 [4 5]]
-----
[[ 6  8]
 [ 8 10]]
```

1.2.3 Operaciones matemáticas generadas en NumPy

Una vez visto como podemos operar Arrays entre ellos, también podemos hacer operaciones matemáticas del lenguaje matemático ordinario junto con los arrays.

A continuación se citan algunas:

- ***np.sqrt()***: Realiza la raíz cuadrada
- ***np.sin()***: Realiza el seno (trigonometría)
- ***np.cos()***: Realiza el coseno (trigonometría)
- ***np.tan()***: Realiza el tangente (trigonometría)
- ***np.log()***: Realiza el logaritmo
- ***np.exp()***: Realiza el exponencial

1.3 Operaciones de agregación y estadística

NumPy nos permite calcular valores agregados o estadísticos sobre un array, como la suma de sus elementos o la media.

Podemos realizar estas operaciones tomando todos los elementos del array o por filas o columnas en una matriz

A continuación veremos las distintas operaciones que puedes realizar:

- `np.sum()`: Realiza el sumatorio de todos los elementos del array
- `np.mean()`: Realiza la media de todos los elementos del array
- `np.std()`: Realiza la desviación estándar del array
- `np.min()`: Indica el valor mínimo del array
- `np.max()`: Indica el valor máximo del array
- `np.median()`: Realiza la mediana de todos los elementos del array.
- `np.cumsum`: Realiza el acumulado de todos los elementos del array

Además puedes realizar estos mismos cálculos **por columna o por fila** con las siguientes estructuras:

- `np.funcion estadística("matriz", axis = 0)`: Realiza el cálculo o indica el resultado de los elementos de cada columna del array
- `np.funcion estadística("matriz", axis = 1)`: Realiza el cálculo o indica el resultado de los elementos de cada fila del array

1.3.1 Sumatorios de arrays

- `np.sum()`: Realiza el sumatorio de los elementos del array

```
[20]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
sumatorio = np.sum(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el sumatorio total da como resultado {sumatorio}")
```

Mi matriz esta compuesta de estos numeros

```
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el sumatorio da como resultado 124

- `np.sum("matriz", axis = 0)`: Realiza el sumatorio de los elementos de cada columna del array

```
[21]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
sumatorio = np.sum(my_matriz, axis = 0)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el sumatorio de las columnas da como resultado {sumatorio}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el sumatorio da como resultado [60 64]

- `np.sum("matriz", axis = 1)`: Realiza el sumatorio de los elementos de cada fila del array

```
[22]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
sumatorio = np.sum(my_matriz, axis = 1)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el sumatorio de las filas da como resultado {sumatorio}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el sumatorio da como resultado [25 29 33 37]

1.3.2 Medias de arrays

- `np.mean()`: Realiza la media de todos los elementos del array

```
[23]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
media = np.mean(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" la media total da como resultado {media}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

la media total da como resultado 15.5

- `np.mean("matriz", axis = 0)`: Realiza la media de los elementos de cada columna del array

```
[24]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
media = np.mean(my_matriz, axis = 0)
print (f" Mi matriz esta compuesta de estos numeros",
        f"\n{my_matriz}")
print(40*"---")
print(f" la media de las columnas da como resultado {media}")
```

Mi matriz esta compuesta de estos numeros

```
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

la media de las columnas da como resultado [15. 16.]

- *np.mean("matriz" , axis = 1)*: Realiza la media de los elementos de cada fila del array

```
[25]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
media = np.mean(my_matriz, axis = 1)
print (f" Mi matriz esta compuesta de estos numeros",
        f"\n{my_matriz}")
print(40*"---")
print(f" la media de las filas da como resultado {media}")
```

Mi matriz esta compuesta de estos numeros

```
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

la media de las filas da como resultado [12.5 14.5 16.5 18.5]

1.3.3 Resto de operaciones estadísticas mencionadas

- *np.std()*: Realiza la desviación estándar del array

```
[27]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
std = np.std(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
        f"\n{my_matriz}")
print(40*"---")
print(f" la desviación estándar da como resultado {std}")
```



```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

la desviacion estandar da como resultado 2.29128784747792

- *np.median()*: Realiza la mediana de todos los elementos del array.

```
[34]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
mediana = np.median(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" la mediana da como resultado {mediana}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

la mediana da como resultado 15.5

- *np.cumsum*: Realiza el acumulado de todos los elementos del array

```
[35]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
acumulado = np.cumsum(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el acumulado da como resultado {acumulado}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el acumulado da como resultado [12 25 39 54 70 87 105 124]

Min

- *np.min()*: Indica el valor minimo del array

```
[28]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
mini = np.min(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor minimo es {mini}")
```

```

Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]

```

```
-----
el valor minimo es 12
```

- *np.min(matriz , axis = 0)*: Indica el valor minimo por columna del array

```
[29]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
mini = np.min(my_matriz, axis = 0 )
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor minimo de las columnas es {mini}")
```

```

Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]

```

```
-----
el valor minimo de las columnas es [12 13]
```

- *np.min(matriz , axis = 1)*: Indica el valor minimo por fila del array

```
[30]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
mini = np.min(my_matriz, axis = 1 )
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor minimo de las filas es {mini}")
```

```

Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]

```

```
[16 17]
[18 19]]
```

el valor minimo de las filas es [12 14 16 18]

Max

- *np.max()*: Indica el valor maximo del array

```
[31]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
maxi = np.max(my_matriz)
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor minimo es {maxi}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el valor minimo es 19

- *np.max(matriz , axis = 0)*: Indica el valor maximo por columna del array

```
[32]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
maxi = np.max(my_matriz, axis = 0 )
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor maximo de las columnas es {maxi}")
```

```
Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]
```

el valor maximo de las columnas es [18 19]

- *np.max(matriz , axis = 1)*: Indica el valor maximo por fila del array

```
[33]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
```

```

maxi = np.max(my_matriz, axis = 1 )
print (f" Mi matriz esta compuesta de estos numeros",
      f"\n{my_matriz}")
print(40*"---")
print(f" el valor maximo de las filas es  {maxi}")

```

```

Mi matriz esta compuesta de estos numeros
[[12 13]
 [14 15]
 [16 17]
 [18 19]]

```

```

-----
el valor maximo de las filas es  [13 15 17 19]

```

1.4 Operaciones de Algebra

Por ultimo, NumpyPy introduce tambien operadores de algebra lineal para la transformacion de los datos de los vectores y matrices.

Uno que ya hemos visto es el *metodo transponedor* o **metodo T** que permite alternar los indices de las filas y columnas de un array, para dar un array con los indices intercambiados (filas por columnas y viceversa)

“nombre matriz”.transpose()

Metodo abreviado: ***“nombre matriz”.T***

```

[40]: import numpy as np

my_matriz = np.arange( 12,20).reshape (4,2)
transponer = my_matriz.transpose()

print (f" Mi matriz original era asi {my_matriz.shape} ",
      f"\n{my_matriz}")
print(40*"---")
print(f" Transpuesta se convierte en una matriz {transponer.shape} quedando asi:
      ↪", f"\n{transponer}")

```

```

Mi matriz original era asi (4, 2)
[[12 13]
 [14 15]
 [16 17]
 [18 19]]

```

```

-----
Transpuesta se convierte en una matriz (2, 4) quedando asi:
[[12 14 16 18]
 [13 15 17 19]]

```

Pero existen otros metodos que podemos utilizar con los arrays, tal y como es *la matriz inversa* y *la traza*

La **matriz inversa** puede aplicarse unicamente a matrices cuadradas (ej: (2,2) (3,3)...) utilizada en metodos de calculo algebratico como la camapana de Gauss, se expresa asi:

np.linalg.inv ("nombre matriz")

La **traza** es la suma de los elementos de la diagonal principal de una matriz cuadrada.

Se expresa asi:

"nombre matriz".trace()

```
[44]: import numpy as np

my_matriz = np.arange( 12,28).reshape (4,4)
inv = np.linalg.inv(my_matriz)

print (f" Mi matriz original era asi {my_matriz.shape} ",
      f"\n{my_matriz}")
print(40*"---")
print(f" La matrix inversa tiene este tamano {inv.shape} quedando asi:",
      f"\n{inv}")
```

```

Mi matriz original era asi (4, 4)
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]
-----
La matrix inversa tiene este tamano (4, 4) quedando asi:
[[ 1.87649984e+14  0.00000000e+00 -5.62949953e+14  3.75299969e+14]
 [ 4.12829966e+15 -9.00719925e+15  5.62949953e+15 -7.50599938e+14]
 [-8.81954927e+15  1.80143985e+16 -9.57014921e+15  3.75299969e+14]
 [ 4.50359963e+15 -9.00719925e+15  4.50359963e+15 -0.00000000e+00]]
```

```
[59]: import numpy as np

my_matriz = np.arange( 12,28).reshape (4,4)
traza = my_matriz.trace()

print (f" Mi matriz original era asi {my_matriz.shape} ",
      f"\n{my_matriz}")
print(40*"---")
print(f" La traza tiene este tamano {traza.shape} quedando asi:", f"\n{traza}")
```

```

Mi matriz original era asi (4, 4)
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]
-----
```

La traza tiene este tamaño () quedando así:
78

Uno de los más interesantes es el *producto interior de las matrices* que tiene una diferencia sustancial:

Realiza la multiplicación de los elementos de 2 arrays de mismo tamaño, dando como resultado **la suma de los productos de los elementos de sus columnas**

Por tanto si realizas esta operación en arrays con 1 dimensión (vectores), te dará como resultado **un número** y si lo realizas con arrays de más dimensiones (matrices), **estas deben tener el mismo tamaño**, devolverá un resultado con **la suma de sus columnas en la misma forma de los arrays**

Se expresará así:

`"array1".dot("array2")`

Has de tener en cuenta:

- Que el producto interior **no es conmutativo** por lo que has de tener claro en cuál calculas el que.

No es conmutativo por lo que va a hacer es coger el valor de la columna x y fila x del array que va a multiplicar (`"array1".dot`) y multiplica por **cada uno de los elementos de sus respectivas columnas y filas** y una vez los tiene, **los suma**.

- Las matrices no tienen que ser cuadradas **si quieres operar con 1 matriz de 1 dimensión**

```
[52]: import numpy as np

my_matriz_1 = np.arange( 12,28).reshape (4,4)
my_matriz_2 = np.arange( 0,16).reshape (4,4)
x_int = my_matriz_1.dot(my_matriz_2)

print (f" las matrices originales eran así {my_matriz_1.shape} ",
      f"\n{my_matriz_1}" ,
      f"\n{my_matriz_2}")
print(40*"~")
print(f" el producto interior tiene este tamaño {x_int.shape} quedando así:",
      f"\n{x_int}")
```

```
las matrices originales eran así (4, 4)
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

el producto interior tiene este tamaño (4, 4) quedando así:

```

[[344 398 452 506]
 [440 510 580 650]
 [536 622 708 794]
 [632 734 836 938]]

```

```

[53]: import numpy as np

my_matriz_1 = np.arange( 12,28).reshape (4,4)
my_matriz_2 = np.arange( 0,16).reshape (4,4)
x_int_2 = my_matriz_2.dot(my_matriz_1)

print (f" las matrices originales eran asi {my_matriz_1.shape} ",
      f"\n{my_matriz_1}" ,
      f"\n{my_matriz_2}")
print(40*"---")
print(f" el producto interior tiene este tamano {x_int_2.shape} quedando asi:",
      f"\n{x_int_2}")

```

```

las matrices originales eran asi (4, 4)
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
-----
el producto interior tiene este tamano (4, 4) quedando asi:
[[ 128  134  140  146]
 [ 416  438  460  482]
 [ 704  742  780  818]
 [ 992 1046 1100 1154]]

```

```

[58]: import numpy as np

my_matriz_1 = np.arange( 0,4).reshape (2,2)
my_vector = np.array([1,2])
x_int_2 = my_matriz_1.dot(my_vector)

print (f" las matrices originales eran asi {my_matriz_1.shape} ",
      f"\n{my_matriz_1}" ,
      f"\n{my_vector}")
print(40*"---")
print(f" el producto interior tiene este tamano {x_int_2.shape} quedando asi:",
      f"\n{x_int_2}")

```

```

las matrices originales eran asi (2, 2)
[[0 1]

```

$\begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix}$

el producto interior tiene este tamaño (2,) quedando así:
 $\begin{bmatrix} 2 & 8 \end{bmatrix}$